

search :

- API Security Headers
- Interactive API Documentation
- Automated Testing

1-API Security Headers:

تعد واجهات برمجة التطبيقات (APIs) من أهم وسائل التواصل بين التطبيقات والخوادم، ولذلك فإن تأمينها يعتبر أمراً بالغ الأهمية. تستخدم **Security Headers** لحماية البيانات أثناء انتقالها بين العميل (Client) والخادم (Server)، كما تمنع العديد من الهجمات الإلكترونية.

ما هي API Security Headers؟

هي مجموعة من رؤوس (HTTP Headers) التي يرسلها الخادم مع الاستجابة (Response) لتحديد سياسات الأمان التي يجب أن يلتزم بها المتصفح أو التطبيق.

الغرض منها: حماية البيانات.

منع الهجمات الإلكترونية.

تحسين خصوصية المستخدم.

التحكم في كيفية تعامل المتصفح مع المحتوى.

أهمية API Security Headers

- منع هجمات XSS.
- منع Clickjacking.
- حماية ملفات تعريف الارتباط Cookies.
- فرض استخدام HTTPS.
- منع تسريب المعلومات الحساسة. -زيادة مستوى أمان التطبيق.

أشهر Security Headers

1. (Strict-Transport-Security (HSTS

يجبر المتصفح على استخدام HTTPS فقط.

الفوائد: منع هجمات Man-in-the-Middle. منع استخدام HTTP.

2. (Content-Security-Policy (CSP

يحدد المصادر المسموح بتحميل الملفات منها.

الفوائد: تقليل هجمات XSS. - منع تحميل ملفات JavaScript من مواقع غير موثوقة.

3. X-Content-Type-Options

يمنع المتصفح من تخمين نوع الملف.

الفوائد: منع تنفيذ ملفات ضارة.

4 -X-Frame-Options

يمنع عرض الموقع داخل iframe

الفوائد: منع هجمات Clickjacking.

5. Referrer-Policy

يحدد كمية المعلومات المرسلة مع الطلبات

الفوائد: حماية خصوصية المستخدم.

6. Permissions-Policy يعطل بعض خصائص المتصفح.

الفوائد: منع الوصول غير المصرح به للكاميرا والميكروفون.

7. Cache-Control يتحكم في تخزين البيانات.

الفوائد: منع تخزين البيانات الحساسة.

8. Set-Cookie Security Flags:

```
Set-Cookie: session=123;  
Secure;  
HttpOnly;  
SameSite=Strict
```

الخيارات:

Secure

• إرسال الكوكيز عبر HTTPS فقط.

HttpOnly منع JavaScript من الوصول للكوكيز.

SameSite منع هجمات CSRF.

هجمات تمنعها Security Headers

- XSS
- CSRF
- Clickjacking
- MIME Sniffing

- Session Hijacking
- MITM

أفضل الممارسات

- استخدام HTTPS دائماً.
- تفعيل HSTS.
- استخدام CSP.
- حماية Cookies.
- تعطيل التخزين المؤقت للبيانات الحساسة.
- اختبار الرؤوس دورياً.

• 2-Interactive API Documentation

توثيق واجهة برمجة التطبيقات هو دليل يشرح كيفية استخدام الـ API. أما **Interactive API Documentation** فهو توثيق يسمح للمطور بتجربة الـ API مباشرة من المتصفح دون الحاجة لكتابة برنامج.

أهمية Interactive Documentation

- تسهيل فهم الـ API.
- تقليل الأخطاء.
- تسريع عملية التطوير.
- تجربة الطلبات مباشرة.
- اختبار جميع Endpoints.

أشهر الأدوات

1. Swagger UI الأكثر استخداماً.

المميزات: واجهة رسومية. - تجربة الطلبات. - عرض Responses. - دعم OAuth.

OpenAPI Specification 2

معيّار عالمي لوصف APIs. يستخدم ملفات YAML أو JSON.

Redoc 3

يعرض التوثيق بشكل احترافي. المميزات: تصميم أنيق. -مناسب للمشاريع الكبيرة -سريع.

Postman Documentation 4 يوفر:

- إنشاء Documentation تلقائياً.
- مشاركة التوثيق. -تجربة الطلبات.

مكونات Interactive Documentation

- Endpoints
- HTTP Methods
- Parameters
- Headers
- Request Body
- Response
- Status Codes
- Authentication
- Examples

فوائد Interactive Documentation

- تقليل وقت التعلم.
- سهولة اختبار API.

- توحيد طريقة الاستخدام.
- تحسين تجربة المطور.
- تقليل الدعم الفني.

أفضل الممارسات

- تحديث التوثيق باستمرار.
- توفير أمثلة.
- توضيح الأخطاء.
- شرح Authentication.
- دعم البحث داخل التوثيق.

3. Automated Testing

هو استخدام برامج تقوم باختبار التطبيق تلقائياً دون تدخل يدوي للتأكد من أن جميع الوظائف تعمل بشكل صحيح.

أهمية Automated Testing

- اكتشاف الأخطاء بسرعة.
- تقليل الوقت.
- تقليل التكلفة.
- تحسين الجودة.
- تسهيل تطوير المشروع.

أنواع الاختبارات

Unit Testing

اختبار كل دالة منفردة.

أدوات:

- JUnit •
- NUnit •
- pytest •

Integration Testing

اختبار تفاعل عدة مكونات معاً - API مع قاعدة البيانات.

System Testin

Regression Testing التأكد من أن التعديلات الجديدة لم تؤثر على الوظائف القديمة.

API Testing

اختبار جميع Endpoints. يشمل:

- Authentication •
- Authorization •
- Response •
- الأداء •

Performance Testing

يقيس:

- سرعة الاستجابة.
- عدد المستخدمين.
- استهلاك الموارد.

أدوات:

- JMeter
- Gatling
- k6

Security Testing

يتحقق من:

- SQL Injection
- XSS
- Authentication
- Authorization

أشهر أدوات Automated Testing

Selenium اختبار واجهات المستخدم.

Postman لاختبار APIs.

Newman تشغيل اختبارات Postman من سطر الأوامر.

JUnit للغة Java.

Pytest للغة Python.

Jest للغة JavaScript.

Cypress لاختبار تطبيقات الويب.

Playwright اختبار المتصفحات الحديثة.

دورة عمل Automated Testing

1. كتابة Test Cases.
2. تنفيذ الاختبارات تلقائياً.
3. مقارنة النتائج بالنتائج المتوقعة.
4. إنشاء تقرير.
5. إصلاح الأخطاء.
6. إعادة الاختبار.

مزايا Automated Testing

- سرعة التنفيذ.
- تقليل الأخطاء البشرية.
- إعادة الاستخدام.
- دعم (Continuous Integration (CI/CD.
- تغطية اختبارية واسعة.

العيوب

- يحتاج وقتاً للإعداد.
 - تكلفة أولية.
 - يحتاج صيانة عند تغيير التطبيق.
 - لا يغني عن الاختبار اليدوي بالكامل.
-

أفضل الممارسات

- كتابة اختبارات واضحة ومستقلة.
- تشغيل الاختبارات مع كل تحديث للكود.
- دمج الاختبارات مع أدوات CI/CD مثل Jenkins أو GitHub Actions.
- تحديث الاختبارات مع أي تغيير في التطبيق.
- التركيز على السيناريوهات الحرجة أولاً.